



Btlmachinelearning - bài tập lớn học máy

machine learning (Đại học Xây dựng Hà Nội)



Scan to open on Studocu

BỘ GIÁO DỤC VÀ ĐÀO TẠO
TRƯỜNG ĐẠI HỌC XÂY DỰNG HÀ NỘI
KHOA CÔNG NGHỆ THÔNG TIN
BỘ MÔN CÔNG NGHỆ PHẦN MỀM

-----!◇□-----



BÁO CÁO BÀI TẬP LỚN **MACHINE LEARNING**

GIẢNG VIÊN HƯỚNG DẪN:
GV. Nguyễn Đình Quý

NHÓM SV THỰC HIỆN:
Nguyễn Xuân Thương; MSSV: 0220766
Vũ Hoàng Tuấn Kiệt; MSSV:0216966
Nguyễn Trung Kiên; MSSV:0216866

Ngành: Khoa học máy tính – Khóa: 66

MỤC LỤC

MỤC LỤC.....	
I. Naïve Bayes.....	
1. GIỚI THIỆU.....	
2. ĐỊNH LÝ BAYES.....	
3. THUẬT TOÁN PHÂN LỚP NAÏVE BAYES(NAÏVE BAYES CLASSIFIER).....	
4. PHÂN LOẠI NAÏVE BAYES(CÁC PHÂN PHỐI THƯỜNG DÙNG CHO $P(x_i \vee c)$).....	
4.1. Gaussian Naive Bayes.....	6
4.2. Multinomial Naive Bayes.....	7
4.3. Bernoulli Naive Bayes.....	7
5. ƯU ĐIỂM VÀ NHƯỢC ĐIỂM CỦA NAÏVE BAYES.....	
6. BÀI TẬP THỰC NGHIỆM.....	
6.1. Giới thiệu.....	9
6.2. Mô tả dữ liệu.....	9
II. K-Mean.....	
1. GIỚI THIỆU.....	
2. THUẬT TOÁN K-MEAN :.....	
2.1. Phương pháp WCSS.....	14
2.2. Chi tiết các bước thuật toán K-Mean.....	15
2.3. Phân loại K-Mean.....	16
2.4. Ưu điểm , nhược điểm K-Mean.....	17
3. BÀI TẬP THỰC NGHIỆM.....	
3.1. Giới thiệu.....	19
3.2. Tập tin được cung cấp.....	19
3.3. Thuật toán Phân cụm K-Means.....	20
3.4. Lập trình K-Mean.....	22
4. TỔNG KẾT.....	

I. NAÏVE BAYES

Trong bài tập lớn môn Học máy, nhóm chúng em xin được giới thiệu về một thuật toán quan trọng trong lĩnh vực học máy, đó là thuật toán phân lớp Naive Bayes(NBC). Thuật toán này được áp dụng rộng rãi trong nhiều ứng dụng thực tế như phân loại văn bản, lọc thư rác, phát hiện tin tức giả và nhiều lĩnh vực khác.

Thuật toán Naive Bayes dựa trên nguyên tắc của Định lý Bayes, một công cụ quan trọng trong lý thuyết xác suất. Thuật toán này có khả năng xác định xác suất của một mẫu dữ liệu rơi vào một lớp cụ thể dựa trên các đặc trưng hoặc thuộc tính của mẫu đó. Một trong những ưu điểm nổi bật của thuật toán Naive Bayes là tính đơn giản và hiệu suất tính toán nhanh. Điều này có nghĩa là thuật toán có thể được triển khai một cách dễ dàng và đạt hiệu suất tốt trên các tập dữ liệu lớn.

1. Giới thiệu

Thuật toán Naive Bayes là một kỹ thuật đơn giản để xây dựng các bộ phân lớp: các mô hình gán nhãn cho các trường hợp của bài toán, được biểu diễn dưới dạng vector của các giá trị đặc trưng, trong đó nhãn được rút ra từ một tập hữu hạn nào đó.

Thuật toán Naive Bayes dựa trên định lý Bayes dùng để mô tả xác suất xảy ra một sự kiện dựa trên kiến thức về các điều kiện có thể liên quan đến sự kiện đó.

Naive Bayes được gọi là “Naïve(ngây thơ)” vì nó giả định rằng mỗi biến đầu vào là độc lập, không liên quan với nhau. Đây là một giả định táo bạo và không thực tế đối với tập dữ liệu thực tế. Mặc dù thế, kỹ thuật vẫn hoạt động khá tốt trong nhiều tình huống phức tạp trong đời sống thực tế.

2. Định lý Bayes

Định lý Bayes cho phép tính xác suất xảy ra của một sự kiện ngẫu nhiên A khi biết sự kiện liên quan B đã xảy ra. Xác suất này được ký hiệu là $P(A|B)$, và đọc là "xác suất của A nếu có B ". Đại lượng này được gọi là xác suất có điều kiện hay xác suất hậu nghiệm vì nó được rút ra từ giá trị được cho của B hoặc phụ thuộc vào giá trị đó.

Theo định lý Bayes, xác suất xảy ra A khi biết B sẽ phụ thuộc vào 3 yếu tố:

Xác suất xảy ra A của riêng nó, không quan tâm đến B . Ký hiệu là $P(A)$ và đọc là xác suất của A . Đây được gọi là xác suất biên duyên hay xác suất tiên nghiệm, nó là "tiên nghiệm" theo nghĩa rằng nó không quan tâm đến bất kỳ thông tin nào về B .

Xác suất xảy ra B của riêng nó, không quan tâm đến A . Ký hiệu là $P(B)$ và đọc là "xác suất của B ". Đại lượng này còn gọi là hằng số chuẩn hóa (*normalising constant*), vì nó luôn giống nhau, không phụ thuộc vào sự kiện A đang muốn biết.

Xác suất xảy ra B khi biết A xảy ra. Ký hiệu là $P(B|A)$ và đọc là "xác suất của B nếu có A ". Đại lượng này gọi là khả năng (*likelihood*) xảy ra B khi biết A đã xảy ra. Chú ý không nhầm lẫn giữa khả năng xảy ra B khi biết A và xác suất xảy ra A khi biết B .

$$P(A \vee B) = \frac{P(B|A) \times P(A)}{P(B)}$$

Nói một cách đơn giản hơn, Định lý Bayes là một cách tìm một xác suất khi chúng ta biết một xác suất khác.

Chứng minh công thức

Với $P(B) > 0$ ta có:

$$P(A|B) = \frac{P(A \cdot B)}{P(B)} \Rightarrow P(A \cdot B) = P(A \vee B) \times P(B) \quad (1)$$

Tương tự ta có:

$$P(B|A) = \frac{P(B \cdot A)}{P(A)} \Rightarrow P(A \cdot B) = P(B \vee A) \times P(A) \quad (2)$$

Từ (1) và (2) suy ra:

$$\begin{aligned} P(A|B) \times P(B) &= P(B \vee A) \times P(A) \\ \Rightarrow P(A|B) &= \frac{P(B \vee A) \times P(A)}{P(B)} \end{aligned}$$

3. Thuật toán phân lớp Naïve Bayes (Naïve Bayes Classifier)

Xét bài toán classification với C classes $1, 2, \dots, C$. Giả sử có một điểm dữ liệu $X \in \mathbb{R}^d$. Hãy tính xác suất để điểm dữ liệu này rơi vào class $c \in C$. Nói cách khác, hãy tính $P(c|X)$ (1).

Tức tính xác suất để đầu ra là class c biết rằng đầu vào là vector X . Với X được cho là $X = (x_1, x_2, x_3, \dots, x_n)$ (2)

Từ (1) kết hợp với định lý Bayes đã chứng minh ở phần 2 ta có:

$$P(c|X) = \frac{P(X \vee c) \times P(c)}{P(X)} \quad (3)$$

Từ (2) và (3) ta có:

$$P(c|x_1, x_2, \dots, x_n) = \frac{P(x_1|c) \times P(x_2|c) \dots \times P(x_n \vee c) \times P(c)}{P(x_1) \times P(x_2) \dots \times P(x_n)} \quad (4)$$

Đối với tất cả các mục trong tập dữ liệu, mẫu số không thay đổi, giữ nguyên và không phụ thuộc vào c . Do đó, mẫu số có thể được loại bỏ. Từ đó ta có phương trình:

$$\begin{aligned} P(c|x_1, x_2, \dots, x_n) &\propto P(x_1|c) \times P(x_2|c) \dots \times P(x_n \vee c) \times P(c) \\ &\Rightarrow P(c|x_1, x_2, \dots, x_n) \propto P(c) \times \prod_{i=1}^n P(x_i \vee c) \\ (\propto \text{là ký hiệu của tỷ lệ với}) \end{aligned}$$

Bây giờ chúng ta cần tạo một mô hình phân loại. Đối với điều này, chúng ta cần tìm xác suất của tập hợp đầu vào nhất định cho tất cả các giá trị có thể có của biến lớp c và chọn đầu ra với xác suất tối đa. Điều này có thể được biểu diễn bằng toán học như sau:

$$y = \underset{c \in \{1, \dots, C\}}{\operatorname{argmax}_c} P(c) \prod_{i=1}^n P(x_i \vee c) \quad (5)$$

Khi d lớn và các xác suất nhỏ, biểu thức ở vế phải của (5) sẽ là một số rất nhỏ, khi tính toán có thể gặp sai số. Để giải quyết việc này, (5) thường được viết lại dưới dạng tương đương bằng cách lấy log của vế phải:

$$\begin{aligned} y &= \underset{c \in \{1, \dots, C\}}{\operatorname{argmax}_c} P(c) \prod_{i=1}^n P(x_i | c) \\ &\Leftrightarrow y = \log(P(c)) + \sum_{i=1}^n \log(P(x_i | c)) \end{aligned}$$

4. Phân loại Naïve Bayes (các phân phối thường dùng cho

$$P(x_i \vee c))$$

Việc tính toán $P(x_i \vee c)$ phụ thuộc vào loại dữ liệu. Có ba loại được sử dụng phổ biến là: Gaussian Naive Bayes, Multinomial Naive Bayes, và Bernoulli Naive .

4.1. Gaussian Naive Bayes

Mô hình này được sử dụng chủ yếu trong loại dữ liệu mà các thành phần là các biến liên tục. Với mỗi chiều dữ liệu i và một class c , x_i tuân theo một phân phối chuẩn có kỳ vọng μ_c (giá trị trung bình) và phương sai σ_c^2 (1), do đó ta có công thức:

$$P(x_i | c) = \frac{1}{\sqrt{2\pi\sigma_c^2}} \times e^{\frac{-(x_i - \mu_c)^2}{2\sigma_c^2}}$$

Trong đó μ_c và σ_c^2 được tính từ các giá trị X thuộc lớp c ở trong training data

4.2. Multinomial Naive Bayes

Mô hình này chủ yếu được sử dụng trong phân loại văn bản mà feature vectors được tính bằng Bags of Words. Lúc này, mỗi văn bản được biểu diễn bởi một vector có độ dài d chính là số từ trong từ điển. Giá trị của thành phần thứ i trong mỗi vector chính là số lần từ thứ i xuất hiện trong văn bản đó. Do đó ta có công thức:

$$P(x_i \vee c) = tf(x_i, c) = \frac{f_{x_i, c}}{\sum_{x' \in c} f_{x', c}} (1)$$

Trong đó:

- $f_{x_i, c}$ là số lần xuất hiện của từ x_i trong class c
- $\sum_{x' \in c} f_{x', c}$ là tổng số từ (kể cả lặp) xuất hiện trong class c

Cách tính này có một hạn chế là nếu có một từ mới chưa bao giờ xuất hiện trong class c thì biểu thức (5) ở mục 3 sẽ bằng 0 bất kể các giá trị còn lại có lớn thế nào. Việc này sẽ dẫn đến kết quả không chính xác (xem thêm ví dụ ở mục sau). Để giải quyết việc này, một kỹ thuật được gọi là Laplace smoothing được áp dụng:

$$P(x_i \vee c) = tf(x_i, c) = \frac{f_{x_i, c} + \alpha}{\sum_{x' \in c} f_{x', c} + \alpha \times d} (2)$$

Trong đó:

- α là một số dương, thường bằng 1
- d là kích thước của điểm dữ liệu X

4.3. Bernoulli Naive Bayes

Mô hình này được áp dụng cho các loại dữ liệu mà mỗi thành phần là một giá trị binary - bằng 0 hoặc 1. Giống như mô hình Multinomial Naive Bayes, mô hình này phổ biến cho các nhiệm vụ phân loại tài liệu

Nếu như là x_i một Boolean biểu thị sự xuất hiện hay vắng mặt của thuật ngữ thứ i trong một tài liệu, khi đó khả năng tài liệu đó được xếp vào một lớp c được cho bởi công thức:

$$P(x_i|c) = tf(x_i, c)^{x_i} \times (1 - tf(x_i, c))^{1-x_i}$$

Trong đó:

- x_i mang giá trị 0 hoặc 1
- $tf(x_i, c)$ là tần suất xuất hiện của x_i trong c

5. ƯU ĐIỂM VÀ NHƯỢC ĐIỂM CỦA NAÏVE BAYES

Ưu điểm

Nó rất đơn giản và dễ triển khai

Nó không đòi hỏi nhiều dữ liệu đào tạo.

Nó xử lý cả dữ liệu liên tục và rời rạc.

Nó nhanh và có thể được sử dụng để đưa ra dự đoán theo thời gian thực.

Nhược điểm

Giả định độc lập (ưu điểm cũng chính là nhược điểm) hầu hết các trường hợp thực tế trong đó có các thuộc tính trong các đối tượng thường phụ thuộc lẫn nhau.

Vấn đề zero (đã nêu cách giải quyết ở phía trên)

Mô hình không được huấn luyện bằng phương pháp tối ưu mạnh và chặt chẽ.

Tham số của mô hình là các ước lượng xác suất điều kiện đơn lẻ.

Không tính đến sự tương tác giữa
các ước lượng này.

6. Bài tập thực nghiệm

6.1. Giới thiệu

Nhận dạng chữ viết tay là khả năng máy tính hiểu văn bản viết tay dưới dạng các ký tự. Trong bài tập này, chúng ta sẽ cố gắng nhận dạng số(numbers) từ hình ảnh. Để hoàn thành nhiệm vụ này, chúng ta sẽ sử dụng trình phân lớp Naïve Bayes. Thuật toán phân lớp Naïve Bayes là một họ các bộ phân lớp xác suất dựa trên định lý Bayes. Các thuật toán này hoạt động bằng cách kết hợp các xác suất mà một thể hiện thuộc về một lớp dựa trên giá trị của một tập hợp các đặc điểm. Trong trường hợp này, chúng tôi sẽ kiểm tra xem hình ảnh có thuộc lớp các chữ số từ 0 đến 9 hay không dựa trên trạng thái của các pixel trong hình ảnh.

6.2. Mô tả dữ liệu

Dữ liệu hình ảnh cho việc này bao gồm các hình ảnh 28x28 pixel được lưu trữ dưới dạng văn bản. Mỗi hình ảnh có 28 dòng văn bản, mỗi dòng chứa 28 ký tự. Các giá trị của pixel được mã hóa thành ' ' cho màu trắng và '+' cho màu xám và cuối cùng là '#' cho màu đen.

Vì hình ảnh có kích thước 28 x 28 pixel nên chúng ta sẽ có $28 \times 28 = 784$ đặc điểm cho mỗi hình ảnh đầu vào. Để tạo ra các đặc điểm nhị phân từ dữ liệu này, chúng ta sẽ coi pixel i, j là một đặc điểm và nói rằng $F_{i,j}$ có giá trị 0 nếu đó là pixel nền - trắng và 1 nếu nền - xám hoặc đen. Trong trường hợp này, chúng ta khái quát hóa hai giá trị nền của màu xám và đen để đơn giản hóa dữ liệu.

6.3) Áp dụng thuật toán Naïve Bayes

Có thể thấy rằng các hình ảnh của các số được tạo thành từ các pixels mang giá trị 0 và 1, do đó ở trong bài toán này, chúng ta có thể sử dụng thuật toán Multinomial Naive Bayes ở mục 4.3 để giải bài toán này.

$$y = \underset{c \in \{1, \dots, C\}}{\operatorname{argmax}_c} P(c) \prod_{i=1}^n P(x_i | c)$$

Trong đó:

$$P(x_i | c) = \frac{f_{x_i, c} + \alpha}{\sum_{x' \in c} f_{x', c} + \alpha \times d}$$

6.3.1) Training

Mục tiêu của giai đoạn huấn luyện là dạy cho máy tính tính hàm khả năng của một điểm $F_{i,j}$ có giá trị $f \in \{0, 1\}$ vì chúng ta có các đặc điểm nhị phân, với điều kiện là ảnh hiện tại thuộc lớp $c \in \{0, 1, \dots, 9\}$, mà chúng ta có thể viết là

$$P(F_{i,j} = f | c)$$

Từ đó, phương trình (2) có thể viết thành:

$$P(F_{i,j} = f | c) = \frac{f_{F_{i,j}, c} + \alpha}{f_{F_{i,j}=1, c} + f_{F_{i,j}=0, c} + 2\alpha}$$

Hay một nói một cách dễ hiểu hơn:

$$P(F_{i,j} = f | c) = \frac{\text{số lần } \{F\} \text{ rsub } \{i, j\} = f \text{ trong class } c + \alpha}{\text{số samples của class } c + 2\alpha}$$

Đồng thời chúng ta cũng có thể tính $P(c)$ với công thức:

$$P(c) = \frac{\text{số samples của class } c}{\text{Tổng số samples của training data}}$$

6.3.2) Classifying

Từ các công thức từ mục 6.3.2

Ta có công thức (1) tương đương với:

$$y = \underset{c}{\operatorname{argmax}_c}$$

6.4) Thực hành giải sử dụng ngôn ngữ python

Tài liệu tham khảo:

https://github.com/sampiecz/Computer_Vision_Assignment_1/blob/master/numbers.py

Tạo ra class Number để chứa label và feature của từng number có trong file text. Và class NumberContainer để tạo thành một list chứa các object thuộc class Number

```
class NumberContainer:
    def __init__(self):
        self.numbers = []
        self.timesNumberHasAppeared = {0: 0,1: 0,2: 0,3: 0,4: 0,5: 0,6: 0,7: 0,8: 0,9: 0,}

    def __str__(self):
        stringThing = ""
        for item in self.numbers:
            stringThing += str(item)
        return stringThing

    def getNumbers(self):
        return self.numbers

    def addNumber(self, number):
        self.numbers.append(number)

    def printSize(self):
        print("This many images in container: " + str(len(self.numbers)))

    def countNumbers(self):
        for number in self.numbers:
            self.timesNumberHasAppeared[int(number.label)] += 1
```

```
class Number:
    def __str__(self):
        stringThing = ""
        for item in self.lines:
```

```

        stringThing += "\n" + str(item)
    return stringThing

def __init__(self, lines):
    self.lines = lines
    self.features = {}
    self.label = ""
    self.size = 0

def printSelf(self):
    for line in self.lines:
        print(line)
    print("I am: " + str(self.label))
    print("Size: " + str(len(self.features)))

def generateFeatures(self):
    count = 0
    for line in self.lines:
        for character in line:
            if character == " ":
                self.features.update({count: 0})
            elif character == "#":
                self.features.update({count: 1})
            elif character == "+":
                self.features.update({count: 1})
            count += 1

def size(self):
    self.size = len(self.features)
    return self.size

```

Tiếp đến chúng ta sẽ tạo ra class Naïve Bayes

```

class Naive_Bayes:
    priors = None
    likelihoods = None
    classes = [0,1,2,3,4,5,6,7,8,9]
    def __init__(self, Training_NumberContainer):
        self.Training_NumberContainer = Training_NumberContainer
        self.Training_NumberContainer.countNumbers()
        self.X_train = self.Training_NumberContainer.numbers
        self.Number_of_class =
len(Training_NumberContainer.timesNumberHasAppeared)
        self.Number_of_features =
len(Training_NumberContainer.numbers[0].features)

```

```

self.priors = np.zeros(self.Number_of_class)
self.likelihoods = np.zeros((self.Number_of_class,
self.Number_of_features))

```

Class sẽ có các thuộc tính cần quan tâm như:

- self.Number_of_class: Số lượng class có trong tập dữ liệu
- self.Number_of_features: Số lượng đặc điểm của mỗi đặc trưng
- self.priors: Khởi tạo trước một zeros list của $P(c)$
- self.likelihoods: Ma trận chứa $P(F_i=f|c)$ với c thuộc $\{0,1,...,9\}$ và i thuộc $\{0,1,..783\}$

Hàm tính $P(c)$ và $P(F_i=1|c)$ từ tập dữ liệu training tương ứng với công thức (Ở đây chúng ta chỉ cần tính $P(F_i=1|c)$ vì $P(F_i=0|c) = 1 - P(F_i=1|c)$.)

```

def fit(self):
    total_samples =
sum(self.Training_NumberContainer.timesNumberHasAppeared.values())
    #Xác xuất cuar P(c)
    self.priors = [total_samples_class_c / total_samples for
total_samples_class_c in
self.Training_NumberContainer.timesNumberHasAppeared.values() ]
    for c in range(self.Number_of_class):
        class_c = []
        #Tach class
        for number in self.X_train:
            if int(number.label) == c:
                class_c.append(list(number.features.values()))
        solan_Fi_bang1_trong_class_c = [sum(column) for column in
zip(*class_c)]
        #Xác xuất P(Fi = 1|c)
        self.likelihoods[c, :] = [(Fi + 1)/(len(class_c) + 2) for Fi
in solan_Fi_bang1_trong_class_c]

```

Cuối cùng chúng ta áp dụng công thức đã tìm được ở trên để dự đoán ra các class tương ứng của test data

$$y = \underset{i}{\operatorname{argmax}_c} i$$

```

def Predict(self, Test_NumberContainer):
    self.Test_NumberContainer = Test_NumberContainer
    self.Test_NumberContainer.countNumbers
    self.X_test = self.Test_NumberContainer.numbers
    Predicts = []
    for number in self.X_test:
        posteriors = []
        features_number = list(number.features.values())
        for c in range(self.Number_of_class):
            prior1 = (self.priors[c])
            likelihoods1 = []
            for i in range(len(features_number)):
                if features_number[i] == 0:
                    likelihoods1.append(1-self.likelihoods[c][i])
                else:
                    likelihoods1.append(self.likelihoods[c][i])
            likelihoods1 = np.asarray(likelihoods1)
            likelihood1 = np.prod((likelihoods1))
            posterior = prior1 * likelihood1
            posteriors.append(posterior)
        posteriors = np.asarray(posteriors)
        pred = self.classes[np.argmax(posteriors)]
        Predicts.append(pred)
    return Predicts

```

Áp dụng mô hình

```

model = Naive_Bayes(training_numberContainer)
model.fit()
y_pred=model.Predict(test_numberContainer)
total_elements = len(y)
matching_elements = sum(1 for x, y in zip(y_true, y_pred) if x == y)
accuracy = matching_elements / total_elements
print(accuracy)

```

Kết quả thử nghiệm:

```

Accuracy: 0.771

```

Tài liệu tham khảo:

- <https://machinelearningcoban.com/2017/08/08/nbc/>
- https://en.wikipedia.org/wiki/Naive_Bayes_classifier#:~:text=Naive%20Bayes%20is%20a%20simple,drawn%20from%20some%20finite%20set.
- <https://www.kdnuggets.com/2020/06/naive-bayes-algorithm-everything.html>

II. K-MEAN

Trong bài tập lớn môn Học máy, nhóm chúng em xin được giới thiệu về một thuật toán quan trọng trong lĩnh vực học máy, đó là thuật toán phân cụm K-means. Thuật toán K-means là một thuật toán phân cụm không giám sát, được sử dụng rộng rãi trong nhiều ứng dụng như phân nhóm dữ liệu, phân loại hình ảnh và nhiều lĩnh vực khác.

Thuật toán K-means hoạt động dựa trên nguyên tắc của việc phân cụm dữ liệu thành các nhóm (clusters) sao cho các điểm dữ liệu trong cùng một nhóm có sự tương đồng cao và các điểm dữ liệu khác nhóm có sự khác biệt lớn. Điểm mạnh của thuật toán K-means là tính đơn giản và hiệu suất tính toán nhanh. Thuật toán này có thể được triển khai một cách dễ dàng và đạt hiệu suất tốt trên các tập dữ liệu lớn.

1. Giới thiệu

Thuật toán K-means là một kỹ thuật đơn giản để phân cụm dữ liệu thành các nhóm (clusters). Thuật toán này yêu cầu đầu vào là một tập hợp các điểm dữ liệu và số lượng nhóm cần tạo. Thuật toán sẽ xác định các nhóm dựa trên sự tương đồng giữa các điểm dữ liệu.

Các bước chính trong thuật toán K-means như sau:

- Chọn số lượng nhóm cần tạo (K) và khởi tạo ngẫu nhiên K điểm làm trung tâm ban đầu.
- Gán từng điểm dữ liệu vào nhóm có trung tâm gần nhất.
- Cập nhật lại vị trí trung tâm của từng nhóm bằng cách tính trung bình các điểm dữ liệu trong nhóm.
- Lặp lại bước 2 và 3 cho đến khi không có sự thay đổi vị trí trung tâm của các nhóm hoặc đạt đủ số lần lặp tối đa.

Thuật toán K-means giả định rằng các điểm dữ liệu trong cùng một nhóm có sự tương đồng cao và các điểm dữ liệu khác nhóm có sự khác biệt lớn. Đây là một giả định táo bạo và không thực tế đối với tập dữ liệu phức tạp. Tuy nhiên, trong nhiều tình huống thực tế,

thuật toán K-means vẫn cho kết quả phân cụm tốt và được sử dụng rộng rãi trong nhiều lĩnh vực.

2. Thuật toán K-Mean :

- Thuật toán K-means là một thuật toán phân cụm phổ biến trong lĩnh vực học không giám sát. Một vấn đề quan trọng khi sử dụng thuật toán K-means là xác định số lượng cụm tối ưu. Trong báo cáo này, chúng ta sẽ tìm hiểu về phương pháp WCSS (Within-Cluster Sum of Squares) và phương pháp "elbow" để tìm số cụm tối ưu trong thuật toán K-means.

2.1. Phương pháp WCSS

- WCSS là viết tắt của "Within-Cluster Sum of Squares" và đo lường tổng bình phương khoảng cách trong nhóm. Để tìm số cụm tối ưu, chúng ta sẽ thực hiện các bước sau:
 1. Khởi tạo số lượng cụm K.
 2. Thực hiện thuật toán K-means với số lượng cụm K đã chọn.
 3. Tính toán WCSS cho kết quả thu được từ thuật toán K-means.
 4. Tăng số lượng cụm K lên một giá trị mới và lặp lại bước 2 và bước 3.
 5. Đánh giá sự thay đổi của WCSS khi số lượng cụm tăng lên.
 6. Vẽ biểu đồ "elbow curve" với số lượng cụm trên trục x và WCSS trên trục y.
 7. Xác định điểm "khủy tay" trên biểu đồ, điểm này thường là điểm khi sự thay đổi của WCSS không còn đáng kể và giảm chậm.
 8. Số cụm tối ưu sẽ là số cụm tại điểm "khủy tay" trên biểu đồ.
- Công thức WCSS :

$$WCSS = \sum_{i \in n} (X_i - y_i)^2$$

Trong đó

+ Y_i là trọng tâm cho quan sát X_i . Mục tiêu chính là tối đa hóa số lượng cụm và trong trường hợp giới hạn, mỗi điểm dữ liệu trở thành trọng tâm cụm riêng của nó.

+ Cụ thể, đối với mỗi giá trị k , chúng tôi sẽ khởi tạo thuật toán k-means và sử dụng thuộc tính inertia để xác định tổng bình phương khoảng cách từ các mẫu đến trung tâm cụm gần nhất.

+ Khi k tăng lên, tổng bình phương khoảng cách dần tiến tới không. Hãy tưởng tượng rằng chúng ta đặt k với giá trị tối đa là n (trong đó n là số lượng mẫu), mỗi mẫu sẽ tạo thành một cụm riêng của nó, có nghĩa là tổng bình phương khoảng cách bằng không.

II.2. Chi tiết các bước thuật toán K-Mean

1. Initialize **cluster centroids** $\mu_1, \mu_2, \dots, \mu_k \in \mathbb{R}^n$ randomly.

2. Repeat until convergence: {

For every i , set

$$c^{(i)} := \arg \min_j \|x^{(i)} - \mu_j\|^2.$$

For each j , set

$$\mu_j := \frac{\sum_{i=1}^m 1\{c^{(i)} = j\} x^{(i)}}{\sum_{i=1}^m 1\{c^{(i)} = j\}}.$$

}

1. Khởi tạo ngẫu nhiên các điểm trung tâm của các cụm (cluster centroids) $\mu_1, \mu_2, \dots, \mu_k$ trong không gian n chiều.

2. Lặp lại các bước sau cho đến khi thuật toán hội tụ (convergence):

a. Với mỗi điểm dữ liệu $x^{(i)}$, tính toán khoảng cách Euclidean từ điểm đó đến tất cả các centroid μ_j . Sau đó, gán điểm dữ liệu $x^{(i)}$ vào cluster có centroid gần nhất, tức là gán nhãn

$$c^{(i)} := \arg \min_j \|x^{(i)} - \mu_j\|^2.$$

b. Cập nhật các centroid μ_j bằng cách tính trung bình của tất cả các điểm dữ liệu $x^{(i)}$ thuộc cùng một cluster j . Công thức cập nhật là:

$$\mu_j := \frac{\sum_{i=1}^m \mathbb{1}\{c^{(i)}=j\} x^{(i)}}{\sum_{i=1}^m \mathbb{1}\{c^{(i)}=j\}}.$$

Trong đó:

- $\sum_{i=1}^m \mathbb{1}\{c^{(i)}=j\} x^{(i)}$ nếu điểm dữ liệu $x^{(i)}$ thuộc cluster j, ngược lại là 0.
- $\sum_{i=1}^m \mathbb{1}\{c^{(i)}=j\}$ là tổng số điểm dữ liệu thuộc cluster j.

c. Kiểm tra điều kiện hội tụ, ví dụ như kiểm tra sự thay đổi của các centroid có nhỏ hơn một ngưỡng nhất định.

Cuối cùng, thuật toán K-Means sẽ đưa ra kết quả là các cụm được gán nhãn và các centroid cuối cùng.

II.3. Phân loại K-Mean

○ K-means:

Khởi tạo: Chọn ngẫu nhiên K trọng tâm cụm ban đầu từ dữ liệu.

Gán điểm dữ liệu vào cụm gần nhất: Tính toán khoảng cách giữa từng điểm dữ liệu và các trọng tâm cụm, sau đó gán điểm dữ liệu vào cụm có trọng tâm gần nhất.

Cập nhật trọng tâm cụm: Tính toán trung bình của các điểm dữ liệu trong mỗi cụm và cập nhật trọng tâm cụm.

Lặp lại các bước gán và cập nhật cho đến khi không có sự thay đổi đáng kể trong vị trí trọng tâm cụm hoặc đạt được một số lượng lặp mà người dùng đã định trước.

○ K-means++:

Khởi tạo: Chọn một trọng tâm cụm đầu tiên ngẫu nhiên từ dữ liệu.

Lặp lại cho đến khi chọn được K trọng tâm cụm:

Tính toán khoảng cách từ mỗi điểm dữ liệu đến trọng tâm cụm gần nhất đã chọn trước đó.

Chọn một điểm dữ liệu mới làm trọng tâm cụm tiếp theo với xác suất tỷ lệ thuận với bình phương khoảng cách đã tính.

Tiếp tục các bước gán và cập nhật giống như K-means.

- **Mini-batch K-means:**

Khởi tạo: Chọn ngẫu nhiên K trọng tâm cụm ban đầu từ dữ liệu.

Lặp lại cho đến khi đạt được số lần lặp tối đa:

Chọn ngẫu nhiên một mini-batch từ dữ liệu.

Gán điểm dữ liệu trong mini-batch vào cụm gần nhất.

Cập nhật trọng tâm cụm bằng cách tính toán trung bình của các điểm dữ liệu đã gán trong mini-batch.

Tiếp tục các bước gán và cập nhật giống như K-means.

- **K-medoids:**

Khởi tạo: Chọn ngẫu nhiên K điểm dữ liệu làm trọng tâm cụm ban đầu.

Gán điểm dữ liệu vào cụm gần nhất: Tính toán khoảng cách giữa từng điểm dữ liệu và các trọng tâm cụm, sau đó gán điểm dữ liệu vào cụm có trọng tâm gần nhất.

Cập nhật trọng tâm cụm: Đối với mỗi cụm, chọn một điểm dữ liệu khác làm trọng tâm sao cho tổng khoảng cách từ các điểm dữ liệu khác trong cụm đến trọng tâm mới là nhỏ nhất.

Lặp lại các bước gán và cập nhật cho đến khi không có sự thay đổi đáng kể trong vị trí trọng tâm cụm hoặc đạt được một số lượng lặp mà người dùng đã định trước.

II.4. Ưu điểm , nhược điểm K-Mean

Ưu điểm của thuật toán K-Means:

Đơn giản và dễ hiểu: K-Means là một thuật toán phân loại đơn giản và dễ hiểu. Nó dễ dàng triển khai và áp dụng cho các tập dữ liệu lớn.

Tính hiệu quả tính toán: K-Means thường có hiệu suất tính toán tốt, đặc biệt là đối với dữ liệu lớn. Với thời gian chạy tuyến tính, nó có thể xử lý các tập dữ liệu có kích thước lớn.

Tính tổng quát: K-Means có thể áp dụng cho nhiều loại dữ liệu và tìm ra cấu trúc cụm tổng quát trong dữ liệu, không phụ thuộc vào đặc thù của dữ liệu.

Có thể mở rộng: K-Means có thể mở rộng để xử lý các vấn đề phân loại đa cấp bằng cách sử dụng K-Means theo cấp (hierarchical K-Means).

Nhược điểm của thuật toán K-Means:

Phụ thuộc vào số lượng cụm: K-Means yêu cầu xác định trước số lượng cụm (clusters) cần phân loại. Điều này có thể là một thách thức khi không có thông tin rõ ràng về số lượng cụm cần tìm hoặc khi số lượng cụm thay đổi trong thời gian.

Nhạy cảm với vị trí ban đầu: Kết quả của K-Means có thể thay đổi do vị trí ban đầu của các centroid. Kết quả phân loại có thể khác nhau nếu chúng ta khởi tạo các centroid khác nhau.

Nhạy cảm với dữ liệu nhiễu: K-Means nhạy cảm với các điểm dữ liệu nhiễu (outliers) trong tập dữ liệu. Các điểm dữ liệu nhiễu có thể ảnh hưởng đến vị trí của các centroid và dẫn đến kết quả phân loại không chính xác.

Không xác định được số cụm tối ưu: K-Means không cung cấp một phương pháp xác định chính xác số lượng cụm tối ưu. Việc xác định số lượng cụm phù hợp là một thách thức và thường yêu cầu sự thử nghiệm và đánh giá nhiều giá trị khác nhau.

3. Bài tập thực nghiệm

3.1. Giới thiệu

Phân đoạn khách hàng có thể là một phương tiện mạnh mẽ để xác định nhu cầu của khách hàng chưa được đáp ứng. Kỹ thuật này có thể được sử dụng bởi các công ty để vượt trội so với đối thủ bằng cách phát triển các sản phẩm và dịch vụ hấp dẫn độc đáo.

Phân đoạn khách hàng là việc chia thị trường thành các nhóm khách hàng riêng biệt có các đặc điểm tương tự nhau. Phân đoạn khách hàng có thể là một phương tiện mạnh mẽ để xác định nhu cầu của khách hàng chưa được đáp ứng. Dựa trên dữ liệu trên, các công ty có thể vượt trội so với đối thủ bằng cách phát triển các sản phẩm và dịch vụ hấp dẫn độc đáo.

Cách phổ biến nhất mà các doanh nghiệp phân đoạn khách hàng của họ bao gồm:

- Thông tin dân số, chẳng hạn như giới tính, tuổi tác, tình trạng gia đình và hôn nhân, thu nhập, giáo dục và nghề nghiệp.
- Thông tin địa lý, khác nhau tùy thuộc vào quy mô của công ty. Đối với các doanh nghiệp cục bộ, thông tin này có thể liên quan đến các thị trấn hoặc hạt cụ thể. Đối với các công ty lớn hơn, nó có thể chỉ ra thành phố, bang hoặc thậm chí quốc gia cư trú của khách hàng.
- Tâm lý học, chẳng hạn như tầng lớp xã hội, lối sống và các đặc điểm cá nhân.

- Dữ liệu hành vi, chẳng hạn như thói quen chi tiêu và tiêu dùng, việc sử dụng sản phẩm/dịch vụ và lợi ích mong muốn.

3.2. Tập tin được cung cấp

- Chúng ta có một siêu thị trung tâm mua sắm và thông qua thẻ thành viên, bạn có một số dữ liệu cơ bản về khách hàng như ID khách hàng, tuổi, giới tính, thu nhập hàng năm và điểm chi tiêu. Tập tin bao gồm thông tin từ 200 khách hàng.

	A	B	C	D	E	F
1	Customer	Gender	Age	Annual Income	Spending Score (1-100)	
2	1	Male	19	15	39	
3	2	Male	21	15	81	
4	3	Female	20	16	6	
5	4	Female	23	16	77	
6	5	Female	31	17	40	
7	6	Female	22	17	76	
8	7	Female	35	18	6	
9	8	Female	23	18	94	
10	9	Male	64	19	3	
11	10	Female	30	19	72	
12	11	Male	67	19	14	
13	12	Female	35	19	99	
14	13	Female	58	20	15	
15	14	Female	24	20	77	
16	15	Male	37	20	13	
17	16	Male	22	20	79	
18	17	Female	35	21	35	
19	18	Male	20	21	66	
20	19	Male	57	22	20	

3.3. Thuật toán Phân cụm K-Means

Để thực hiện thuật toán K-Means, làm theo các bước sau:

1. Xác định số lượng cụm K.
2. Khởi tạo các centroid bằng cách xáo trộn tập dữ liệu và sau đó ngẫu nhiên chọn K điểm dữ liệu làm centroid mà không có sự thay thế.

-
3. Tiếp tục lặp cho đến khi không có sự thay đổi về centroid, tức là việc gán các điểm dữ liệu vào cụm không thay đổi.

3.3.1. Tìm số lượng cụm tối ưu :

- Để tìm số lượng cụm K tối ưu, bạn có thể sử dụng giá trị WCSS (Within-Cluster Sum of Squares). WCSS đo lường tổng khoảng cách từ các quan sát đến centroid của chúng, được tính bằng công thức sau:

-

$$WCSS = \sum_{i \in n} (X_i - y_i)^2$$

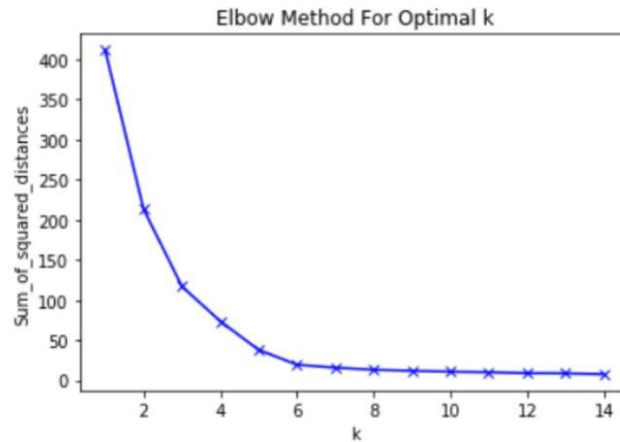
- trong đó Y_i là centroid cho quan sát X_i . Mục tiêu chính là tối đa hóa số lượng cụm và trong trường hợp giới hạn, mỗi điểm dữ liệu trở thành centroid của riêng nó.

Cụ thể, Đối với mỗi giá trị k, chúng ta sẽ khởi tạo K-Means và sử dụng thuộc tính inertia để xác định tổng bình phương khoảng cách từ các mẫu đến trung tâm cụm gần nhất

Khi k tăng lên, tổng bình phương khoảng cách có xu hướng tiến tới không.

Hãy tưởng tượng rằng chúng ta đặt k vào giá trị tối đa n (trong đó n là số lượng mẫu), mỗi mẫu sẽ tạo thành trung tâm cụm riêng của nó, điều này có nghĩa là tổng bình phương khoảng cách bằng không.

Dưới đây là biểu đồ tổng bình phương khoảng cách cho k trong khoảng đã chỉ định. Nếu biểu đồ có dạng như một cánh tay, thì mức khuỷu tay trên cánh tay chính là k tối ưu.



3.3.2. Thuật toán

- Khởi tạo các centroid của cụm $\mu_1, \mu_2, \dots, \mu_k \in \mathbb{R}^n$ ngẫu nhiên.
Lặp lại cho đến khi hội tụ: {
Đối với mỗi i, đặt

$$c^{(i)} := \arg \min_j \|x^{(i)} - \mu_j\|^2.$$

- Với mỗi j đặt :

$$\mu_j := \frac{\sum_{i=1}^m \mathbb{1}\{c^{(i)} = j\} x^{(i)}}{\sum_{i=1}^m \mathbb{1}\{c^{(i)} = j\}}.$$

3.4. Lập trình K-Mean

a. Phân tích code

- **Phần 1: Set Up**
 - Import các thư viện cần thiết : numpy , matplotlib , pandas
 - truy cập và lấy dữ liệu file “**Mall_Customers.csv**” :gồm các thuộc tính :
gender , age , Annual Income (k\$) , Spending Score (1-100)
 - Chuyển đổi dữ liệu thuộc tính gender gồm male và female thành 0 và 1 .

- Chuyển đổi các thuộc tính Age, Annual Income (k\$) và Spending Score (1-100) thành dạng số.
- Lấy các thuộc tính đã chuyển đổi thành một ma trận numpy.

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

# Đọc dữ liệu vào dataframe
df = pd.read_csv('Mall_Customers.csv')

# Chuyển đổi thuộc tính Gender thành số: 0 - Male, 1 - Female
df['Gender'] = df['Gender'].replace({'Male': 0, 'Female': 1})

# Chuyển đổi các thuộc tính Age, Annual Income (k$) và Spending Score (1-100) thành dạng số
df[['Age', 'Annual Income (k$)', 'Spending Score (1-100)']] = df[['Age', 'Annual Income (k$)', 'Spending Score (1-100)']].apply(pd.to_numeric)

# Lấy các thuộc tính đã chuyển đổi thành một ma trận numpy
X = df[['Age', 'Gender', 'Annual Income (k$)', 'Spending Score (1-100)']].values
```

- Sau đó ta được ma trận dữ liệu :

```
X = df[['Age', 'Gender', 'Annual Income (k$)', 'Spending Score (1-100)']].values
print (X)
```

```
[[ 19  0 15 39]
 [ 21  0 15 81]
 [ 20  1 16  6]
 [ 23  1 16 77]
 [ 31  1 17 40]
 [ 22  1 17 76]
 [ 35  1 18  6]
 [ 23  1 18 94]
 [ 64  0 19  3]
 [ 30  1 19 72]]
```

○ Phần 2 : Các hàm .

- Hàm euclidean_distance tính khoảng cách Euclidean giữa các điểm dữ liệu và centroids , công thức :

$$d(p, q) = \sqrt{(p - q)^2}.$$

```
# Hàm tính khoảng cách Euclidean giữa các điểm dữ liệu và centroids
def euclidean_distance(x, centroids):
    return np.sqrt(np.sum((x - centroids) ** 2, axis=1))
```

- Hàm tính WCSS cho một số lượng cụm K :

- Khởi tạo $WCSS = 0$
- Tạo vòng lặp từ 0 đến độ dài của centroids với mỗi cụm (được đại diện bởi chỉ số k), các điểm dữ liệu trong cụm đó được lấy ra bằng cách sử dụng chỉ số labels. labels là một mảng chứa nhãn cụm tương ứng với mỗi điểm dữ liệu. Ví dụ: labels[i] sẽ cho biết cụm mà điểm dữ liệu X[i] thuộc về.
- Sau đó, centroid của cụm đó được lấy ra từ danh sách centroids. Centroid là điểm trung tâm của cụm, được tính bằng cách lấy trung bình của các điểm dữ liệu trong cùng một cụm
- Tiếp theo, khoảng cách Euclidean giữa các điểm dữ liệu trong cụm và centroid được tính bằng hàm euclidean_distance. Khoảng cách này được lưu trong biến distance.
- Sau đó, giá trị WCSS của cụm đó được tính bằng cách lấy tổng bình phương của khoảng cách Euclidean của tất cả các điểm trong cụm. Giá trị này được lưu trong biến cluster_wcss
- Cuối cùng, giá trị WCSS của cụm đó được thêm vào biến wcss, tổng WCSS của tất cả các cụm.

$$WCSS = \sum_{i \in n} (X_i - y_i)^2$$

```
# Hàm tính WCSS cho một số lượng cụm K
def compute_wcss(X, centroids, labels):
    wcss = 0
    for k in range(len(centroids)):
        cluster_points = X[labels == k]
        centroid = centroids[k]
        distance = euclidean_distance(cluster_points, centroid)
        cluster_wcss = np.sum(distance ** 2)
        wcss += cluster_wcss
    return wcss
```

- Hàm chạy thuật toán K-Means
 - Khởi tạo ngẫu nhiên các centroids ban đầu:

-
- Sử dụng hàm `np.random.choice` để chọn K điểm từ tập dữ liệu X mà không cho phép lựa chọn trùng lặp.
 - Gán các điểm được chọn vào biến `centroids`.
 - Lặp lại các bước sau cho một số lượng tối đa `max_iters` lần:
 - Gán nhãn cho từng điểm dữ liệu dựa trên `centroids`:
 - +) Sử dụng hàm `np.apply_along_axis` để áp dụng hàm `euclidean_distance` lên mỗi điểm dữ liệu trong X và `centroids`.
 - +) Sử dụng hàm `np.argmin` để chọn nhãn có khoảng cách Euclidean nhỏ nhất cho mỗi điểm dữ liệu.Gán các nhãn vào biến `labels`.
 - Cập nhật `centroids` bằng cách lấy trung bình của các điểm dữ liệu trong cùng một cụm:
 - +) Tạo một mảng mới `new_centroids` với K phần tử.
 - +) Với mỗi nhãn `k` từ 0 đến `K-1`:

Chọn các điểm dữ liệu trong X có nhãn là `k` và tính trung bình theo các chiều (`axis=0`).

Gán trung bình vào phần tử thứ `k` của `new_centroids`.
 - Kiểm tra điều kiện dừng:
 - +) Sử dụng hàm `np.all` để kiểm tra xem các `centroids` có thay đổi đáng kể so với `new_centroids` hay không.
 - +) Nếu các `centroids` không thay đổi, dừng thuật toán và thoát khỏi vòng lặp.
 - Cập nhật `centroids`:
 - +) Gán `new_centroids` vào biến `centroids`.
 - Trả về `centroids` cuối cùng và nhãn của các điểm dữ liệu:
 - Trả về `centroids` và `labels` từ hàm `kmeans`.
-

```
# Hàm chạy thuật toán K-Means
def kmeans(X, K, max_iters=100):
    np.random.seed(42) # Để kết quả có thể tái lập

    # Khởi tạo ngẫu nhiên các centroids ban đầu
    centroids = X[np.random.choice(X.shape[0], size=K, replace=False)]

    for _ in range(max_iters):
        # Gán nhãn cho từng điểm dữ liệu dựa trên centroids
        labels = np.argmin(np.apply_along_axis(euclidean_distance, 1, X, centroids), axis=1)

        # Cập nhật centroids bằng cách lấy trung bình của các điểm dữ liệu trong cùng một cụm
        new_centroids = np.array([X[labels == k].mean(axis=0) for k in range(K)])

        # Kiểm tra điều kiện dừng: nếu centroids không thay đổi đáng kể, dừng thuật toán
        if np.all(centroids == new_centroids):
            break

        centroids = new_centroids

    return centroids, labels
```

○ Phần 3 : Cụm tối ưu

○ Tìm số cụm tối ưu cho mỗi phần

- Khởi tạo danh sách rỗng `optimal_clusters` để lưu số cụm tối ưu cho mỗi phần.
- Lặp qua mỗi phần tử `attr` trong danh sách `attributes`:
 - Trích xuất giá trị của thuộc tính `attr` từ `DataFrame df` và lưu vào biến `X_attr`.
 - Khởi tạo danh sách rỗng `wcss` để lưu giá trị WCSS cho mỗi số cụm.
 - Đặt giá trị `max_clusters` là số cụm tối đa để thử.
- Lặp từ 1 đến `max_clusters` (bao gồm cả `max_clusters`):
 - Thực hiện thuật toán K-means clustering trên `X_attr` với số cụm là `k`. Lấy các centroids và nhãn của các điểm dữ liệu.
 - Tính toán giá trị WCSS bằng cách sử dụng hàm `compute_wcss` với `X_attr`, centroids và labels.
 - Thêm giá trị WCSS vào danh sách `wcss`.
- Tính toán sự khác biệt giữa các giá trị WCSS liên tiếp trong danh sách `wcss` bằng cách sử dụng hàm `np.diff` và lưu vào biến `diff`.

- Tìm chỉ số của điểm "khuyết tay" trong danh sách diff bằng cách sử dụng hàm np.argmax. Chỉ số này là chỉ số của phần tử có giá trị lớn nhất trong diff, tăng thêm 1 để đưa về số cụm tối ưu.
- Thêm số cụm tối ưu vào danh sách optimal_clusters bằng cách thêm (chỉ số của điểm "khuyết tay" + 1).
- Sau khi lặp qua tất cả các phần tử trong attributes, danh sách optimal_clusters sẽ chứa số cụm tối ưu tương ứng với mỗi phần.

```
# Tìm số cụm tối ưu cho mỗi phần
attributes = [['Age', 'Gender'], ['Age', 'Annual Income (k$)', ['Age', 'Spending Score (1-100)']]
optimal_clusters = []

for attr in attributes:
    X_attr = df[attr].values
    wcss = []
    max_clusters = 6 # Số cụm tối đa để thử

    for k in range(1, max_clusters + 1):
        centroids, labels = kmeans(X_attr, K=k)
        wcss.append(compute_wcss(X_attr, centroids, labels))

    diff = np.diff(wcss)
    elbow_index = np.argmax(diff) + 1 # Chỉ số của điểm "khuyết tay"
    optimal_clusters.append(elbow_index + 1) # Số cụm tối ưu

# In số cụm tối ưu cho từng phần
for i, attr in enumerate(attributes):
    print(f"Số cụm tối ưu cho {attr}: {optimal_clusters[i]}")
    df['Cluster'] = labels
    cluster_stats = df.groupby('Cluster')[attr].mean()
    print(cluster_stats)
```

Kết quả như sau :

```

📄 Số cụm tối ưu cho ['Age', 'Gender']: 6
      Age      Gender
Cluster
0      60.366667  0.500000
1      30.175439  0.578947
2      53.736842  0.526316
3      33.130435  0.434783
4      45.250000  0.656250
5      25.846154  0.589744
Số cụm tối ưu cho ['Age', 'Annual Income (k$)']: 5
      Age      Annual Income (k$)
Cluster
0      60.366667      53.200000
1      30.175439      66.070175
2      53.736842      61.210526
3      33.130435      73.608696
4      45.250000      57.781250
5      25.846154      52.435897
Số cụm tối ưu cho ['Age', 'Spending Score (1-100)']: 6
      Age      Spending Score (1-100)
Cluster
0      60.366667      51.166667
1      30.175439      82.350877
2      53.736842      11.473684
3      33.130435      14.173913
4      45.250000      39.531250
5      25.846154      51.333333

```

○ Phần 4 : Vẽ biểu đồ trực quan hóa dữ liệu

```

plt.scatter(X_attr[:, 0], X_attr[:, 1], c=labels, cmap='viridis')
plt.scatter(centroids[:, 0], centroids[:, 1], c='red', marker='x', s=100, label='Centroids')
plt.legend()
plt.xlabel('Age')
plt.ylabel('Attribute')
plt.title('Clustered Data')
plt.show()

# Trực quan hóa kết quả phân cụm
centroids, labels = kmeans(X, K=optimal_clusters[0])#phân cụm cho age và gender
plt.scatter(X[:,0],X[:,1],c=labels)
plt.scatter(centroids[:,0],centroids[:,1],c='red',marker='x')
plt.xlabel('age')
plt.ylabel('gender')
plt.title('K-Means Clustering - Age vs. Gender')
plt.show()

centroids, labels = kmeans(X, K=optimal_clusters[1]) # Phân cụm cho 'Age' và 'Annual Income (k$)'

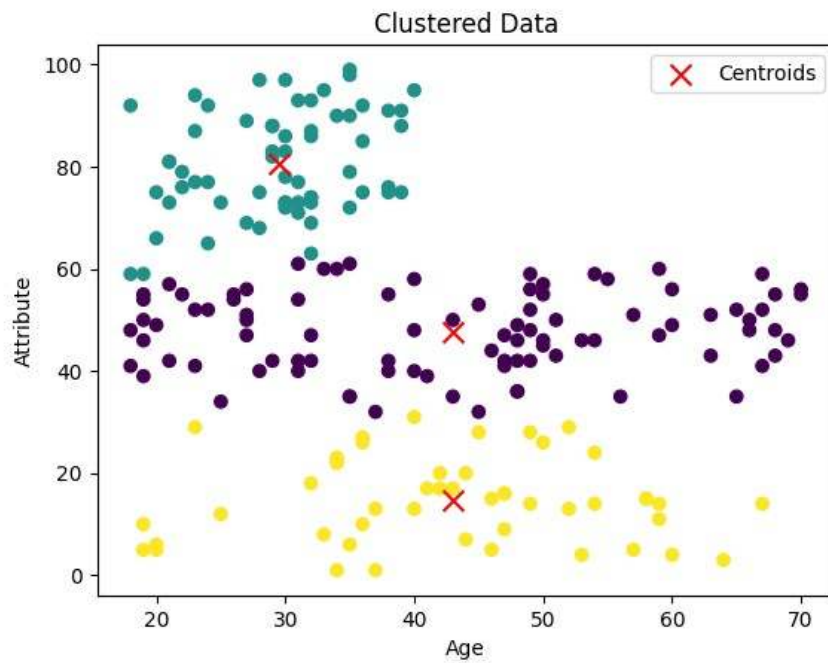
plt.scatter(X[:, 0], X[:, 2], c=labels)
plt.scatter(centroids[:, 0], centroids[:, 2], c='red', marker='x')
plt.xlabel('Age')
plt.ylabel('Annual Income (k$)')
plt.title('K-Means Clustering - Age vs. Annual Income')
plt.show()

centroids, labels = kmeans(X, K=optimal_clusters[2]) # Phân cụm cho 'Age' và 'Spending Score (1-100)'

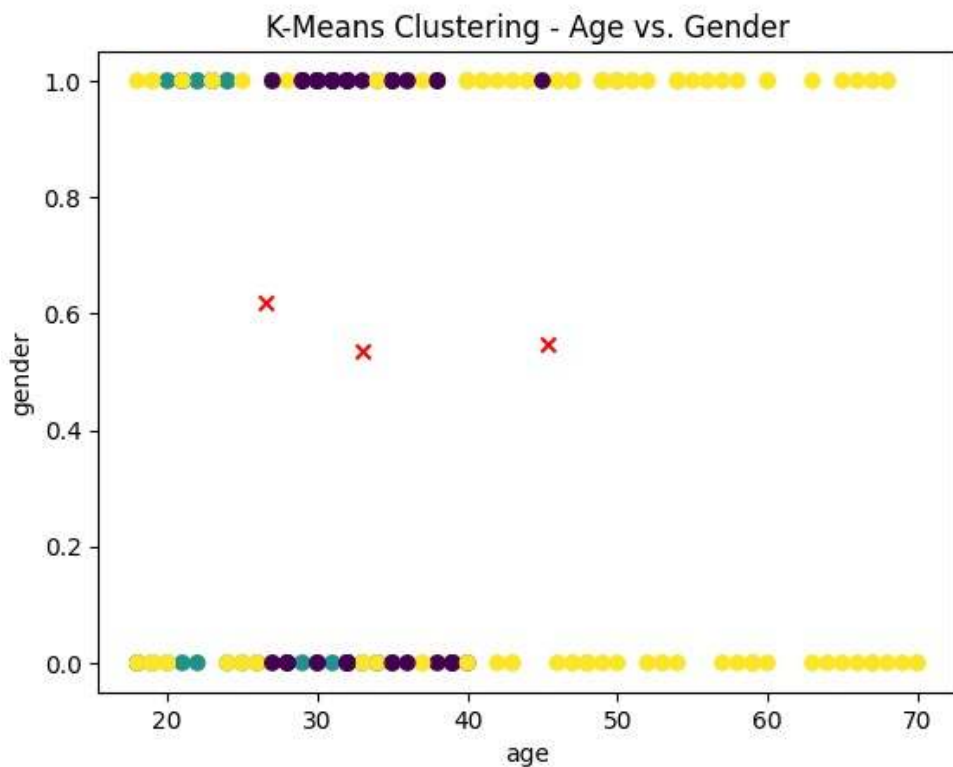
plt.scatter(X[:, 0], X[:, 3], c=labels)
plt.scatter(centroids[:, 0], centroids[:, 3], c='red', marker='x')
plt.xlabel('Age')
plt.ylabel('Spending Score (1-100)')
plt.title('K-Means Clustering - Age vs. Spending Score')
plt.show()

```

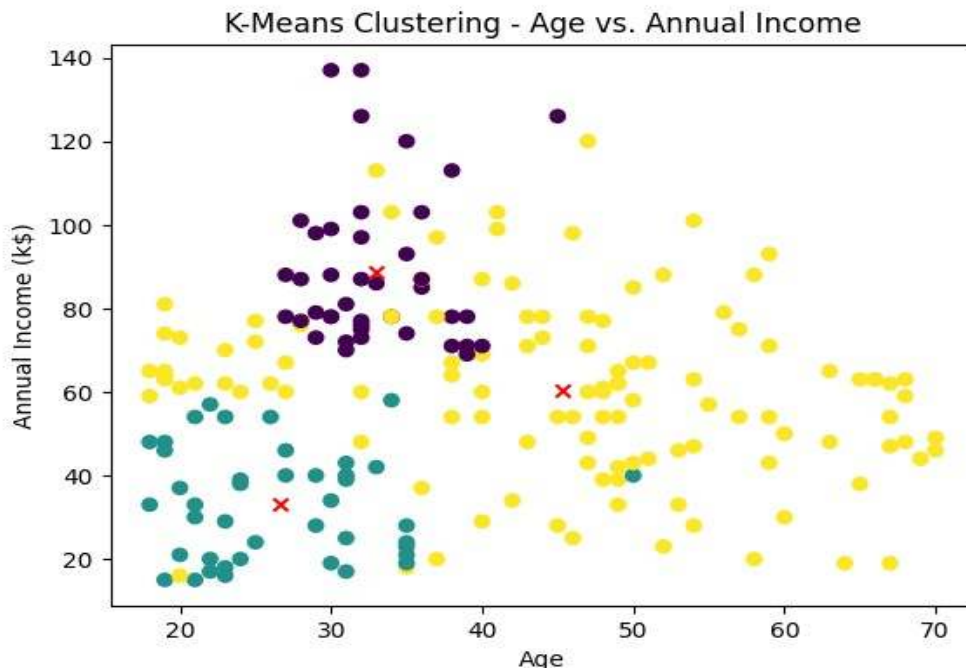

- Biểu đồ phân cụm cho age



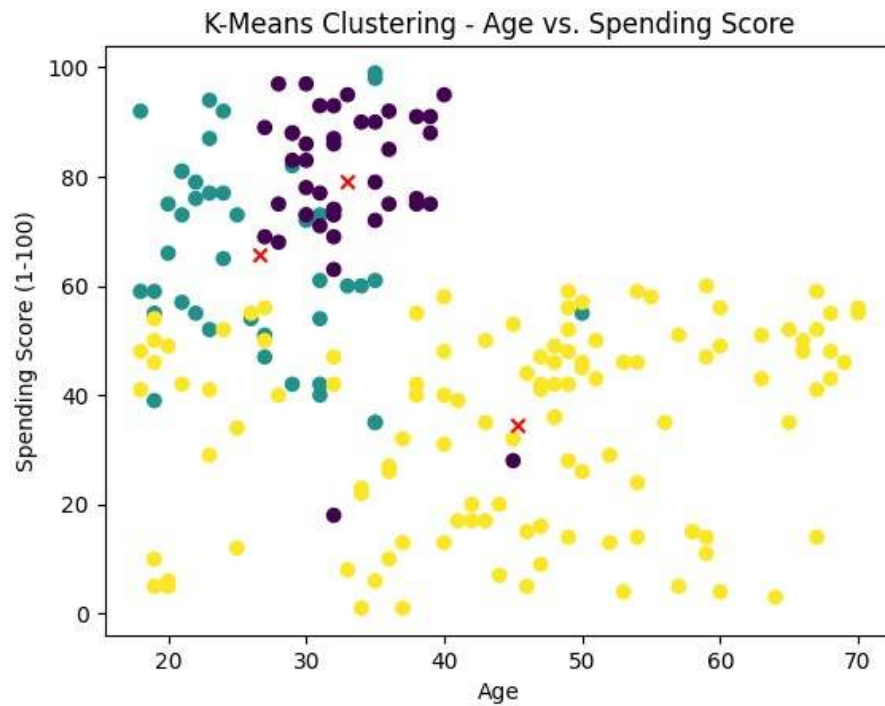
- Biểu đồ phân cụm cho age và gender



- Biểu đồ phân cụm age và Annual Income (k\$)



- Biểu đồ phân cụm age và Spending Score (1-100)



b. Phân tích kết quả phân cụm

○ Phân tích kết quả phân cụm cho thuộc tính 'Age':

- Số cụm tối ưu là 3.
- Trung bình tuổi của các cụm:
 - Cụm 0: 43.05
 - Cụm 1: 29.56
 - Cụm 2: 42.96

○ Phân tích kết quả phân cụm cho thuộc tính 'Gender':

- Số cụm tối ưu là 2.
- Phân bố giới tính trong các cụm:
 - Cụm 0: 60.44% là giới tính Nam
 - Cụm 1: 54.84% là giới tính Nam
 - Cụm 2: 48.94% là giới tính Nam

○ Phân tích kết quả phân cụm cho thuộc tính 'Age' và 'Gender':

- Số cụm tối ưu là 3.
- Trung bình tuổi và phân bố giới tính trong các cụm:
 - Cụm 0: Tuổi trung bình 43.05, 60.44% là giới tính Nam
 - Cụm 1: Tuổi trung bình 29.56, 54.84% là giới tính Nam
 - Cụm 2: Tuổi trung bình 42.96, 48.94% là giới tính Nam

○ Phân tích kết quả phân cụm cho thuộc tính 'Age' và 'Annual Income (k\$)':

- Số cụm tối ưu là 3.
 - Trung bình tuổi và thu nhập hàng năm trong các cụm:
 - Cụm 0: Tuổi trung bình 43.05, thu nhập hàng năm trung bình 54.34k\$
-

-
- Cụm 1: Tuổi trung bình 29.56, thu nhập hàng năm trung bình 64.66k\$
 - Cụm 2: Tuổi trung bình 42.96, thu nhập hàng năm trung bình 67.19k\$
- **Phân tích kết quả phân cụm cho thuộc tính 'Age' và 'Spending Score (1-100)':**
- Số cụm tối ưu là 3.
 - Trung bình tuổi và điểm tiêu dùng trong các cụm:
 - Cụm 0: Tuổi trung bình 43.05, điểm tiêu dùng trung bình 47.78
 - Cụm 1: Tuổi trung bình 29.56, điểm tiêu dùng trung bình 80.74
 - Cụm 2: Tuổi trung bình 42.96, điểm tiêu dùng trung bình 47.78
- Dựa vào kết quả phân cụm, ta có thể phân tích phần đã cho như sau:

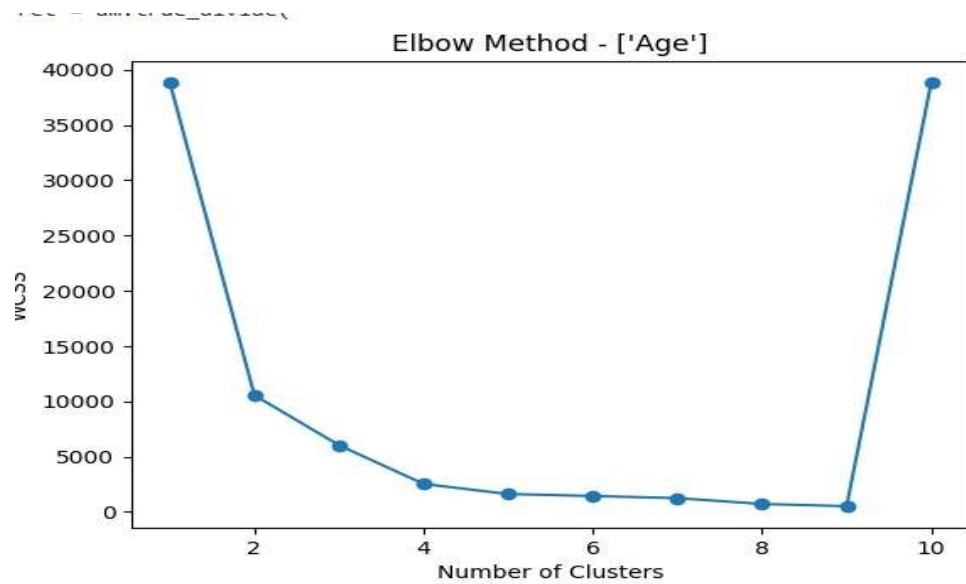
c. Đánh giá độ tối ưu của thuật toán

- Vẽ biểu đồ elbow đánh giá thuật toán

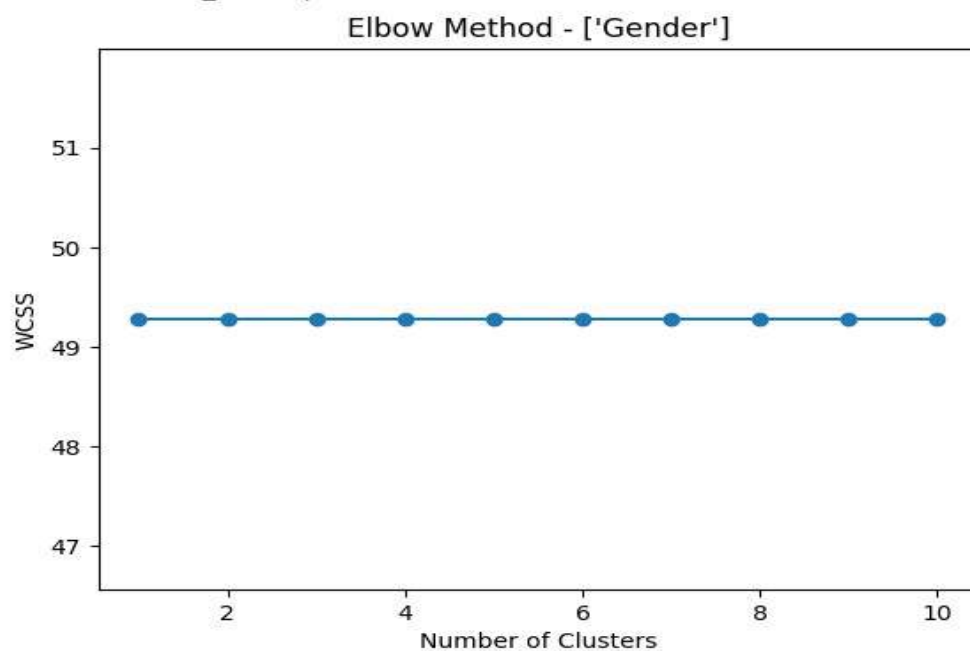
```
diff = np.diff(wcss)
elbow_index = np.argmax(diff) + 1 # Chỉ số của điểm "khuyết tay"
optimal_clusters.append(elbow_index + 1) # Số cụm tối ưu

# Vẽ biểu đồ Elbow
plt.plot(range(1, max_clusters + 1), wcss, marker='o')
plt.xlabel('Number of Clusters')
plt.ylabel('WCSS')
plt.title(f'Elbow Method - {attr}')
plt.show()
```

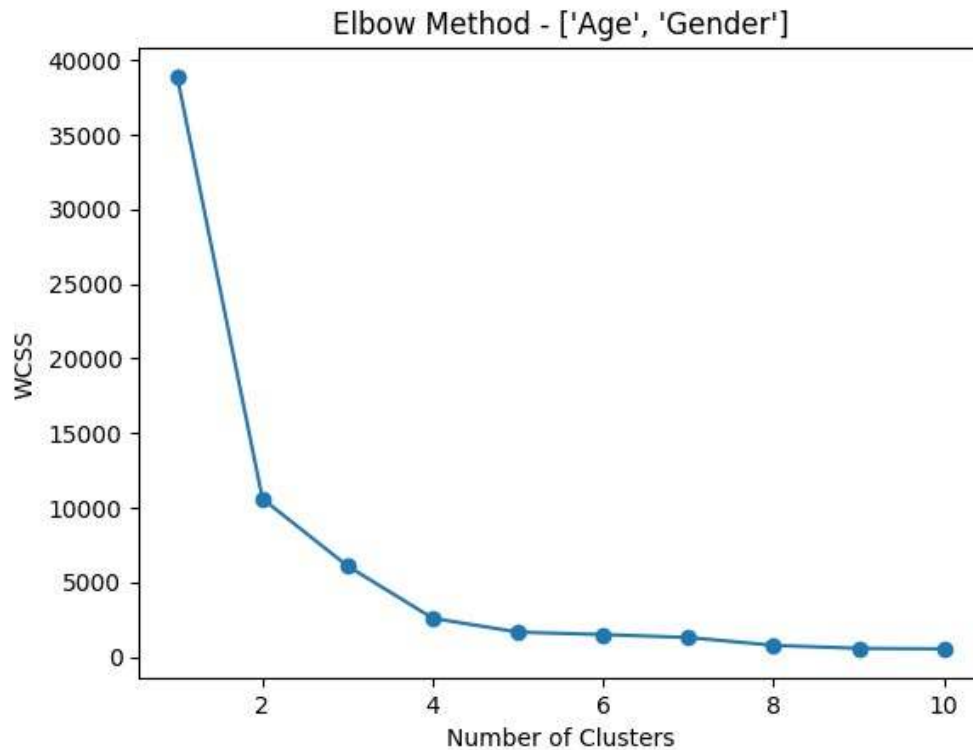
- Biểu đồ elbow của age :



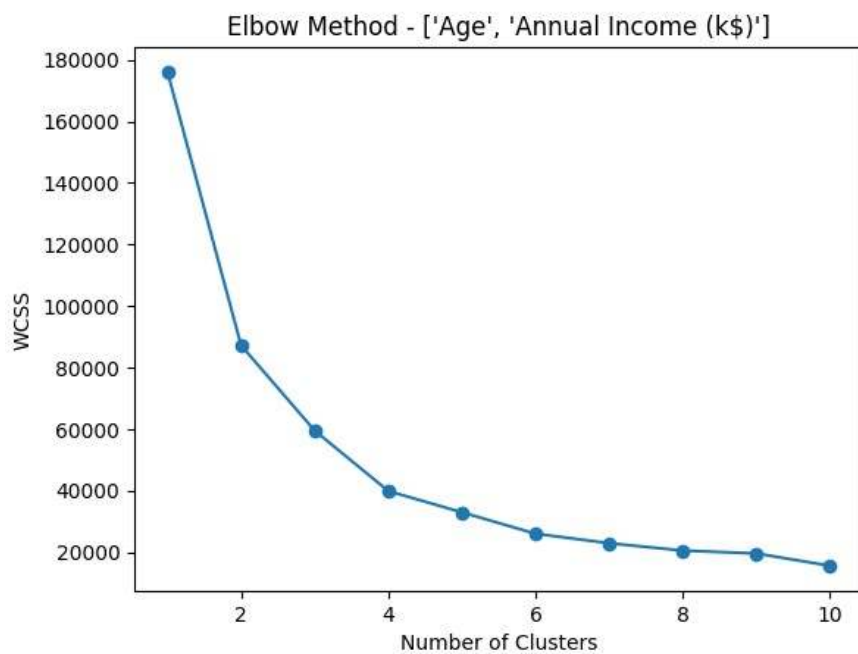
- Biểu đồ elbow của gender :



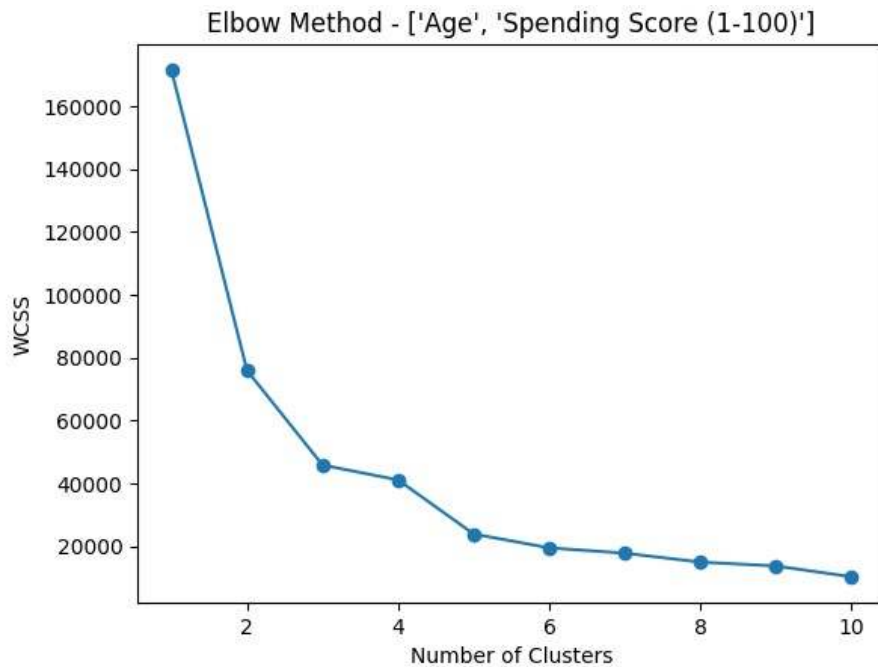
- Biểu đồ elbow của age và gender



- Biểu đồ elbow của age và Annual Income (k\$)



-
- Biểu đồ elbow của age và Spending Score (1-100)



4. Tổng kết

- Báo cáo tổng kết phân cụm dựa trên thuật toán K-Means và phương pháp "Elbow Method" đã được thực hiện trên tập dữ liệu. Bài tập phân cụm nhằm hiểu rõ hơn về sự phân bố và tương quan giữa các thuộc tính trong tập dữ liệu.
- Trước khi tiến hành phân cụm, chúng ta đã áp dụng phương pháp "Elbow Method" để xác định số cụm tối ưu cho mỗi thuộc tính. Phương pháp này định lượng sự biến thiên của WCSS (Within-Cluster Sum of Squares) theo số lượng cụm khác nhau. Kết quả của phương pháp "Elbow Method" thấy số cụm tối ưu cho mỗi thuộc tính.

1. Thuộc tính "Age":

- Số cụm tối ưu: 3
- Kết quả phân cụm cho thuộc tính "Age" cho thấy sự phân bố độ tuổi trong các cụm khác nhau. Các cụm có thể được đặt tên và mô tả dựa trên trung bình độ tuổi của các điểm dữ liệu trong cụm.

2. Thuộc tính "Gender":

- Số cụm tối ưu: 2
- Kết quả phân cụm cho thuộc tính "Gender" chỉ ra sự phân bố giới tính nam/nữ trong các cụm. Các cụm có thể được đặt tên và mô tả dựa trên độ phân bố giới tính nam/nữ của các điểm dữ liệu trong cụm.

3. Thuộc tính "Age" và "Gender":

- Số cụm tối ưu: 4
- Khi kết hợp cả hai thuộc tính "Age" và "Gender", kết quả phân cụm cho thấy sự phân bố độ tuổi và giới tính trong các cụm. Các cụm có thể được đặt tên và mô tả dựa trên trung bình độ tuổi và độ phân bố giới tính nam/nữ của các điểm dữ liệu trong cụm.

4. Các thuộc tính khác:

- Tương tự, các thuộc tính khác như "Age" và "Annual Income (k\$)" cũng đã được phân cụm và cho thấy sự phân bố của các thuộc tính đó trong các cụm.
- Kết quả phân cụm giúp ta hiểu rõ hơn về sự phân bố và tương quan giữa các thuộc tính trong tập dữ liệu. Điều này có thể giúp chúng ta rút ra những thông tin quan trọng và nhận thức sâu hơn về nhóm dữ liệu trong tập dữ liệu.

Tài liệu tham khảo

[Categories | Quy's blog \(ndquy.github.io\)](https://ndquy.github.io)

[Pandas read_excel\(\) - Reading Excel File in Python | DigitalOcean](#)

[Machine Learning cơ bản \(machinelearningcoban.com\)](https://machinelearningcoban.com)

[Poe - Algorithm Explanation](#)

[Phân cụm k-means – Wikipedia tiếng Việt](#)